Web Security

Thierry Sans

Securing the web architecture means securing ...

- The network
- The operating system
- The administration
- The web architecture
- The database
- The web application

Our focus here!

https://owasp.org/Top10/

Top 10:2025 List

1. A01:2025 - Broken Access Control
2. A02:2025 - Security Misconfiguration
3. A03:2025 - Software Supply Chain Failures
4. A04:2025 - Cryptographic Failures
5. A05:2025 - Injection
6. A06:2025 - Insecure Design
7. A07:2025 - Authentication Failures
8. A08:2025 - Software or Data Integrity Failures
9. A09:2025 - Security Logging and Alerting Failures
10. A10:2025 - Mishandling of Exceptional Conditions

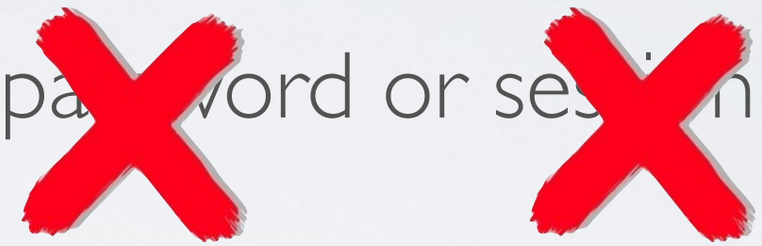
➡ Risks are ranked according to the frequency of discovered security defects, the severity of the uncovered vulnerabilities, and the magnitude of their potential impacts

A04 Cryptographic Failure

Insufficient Transport Protection

How to steal user's credentials

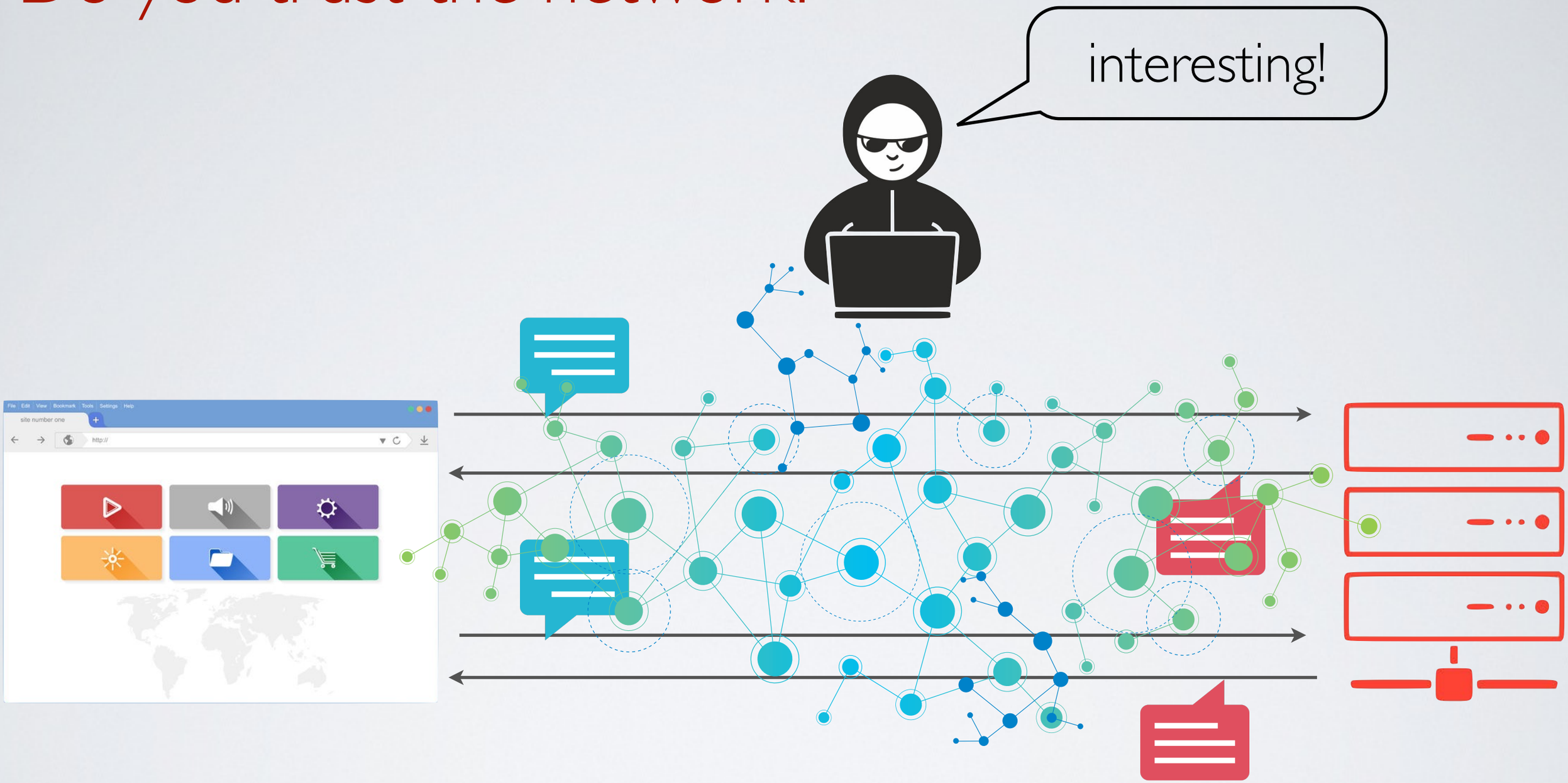
- Brute force the user's password or session ID



- Steal the user's password or session ID

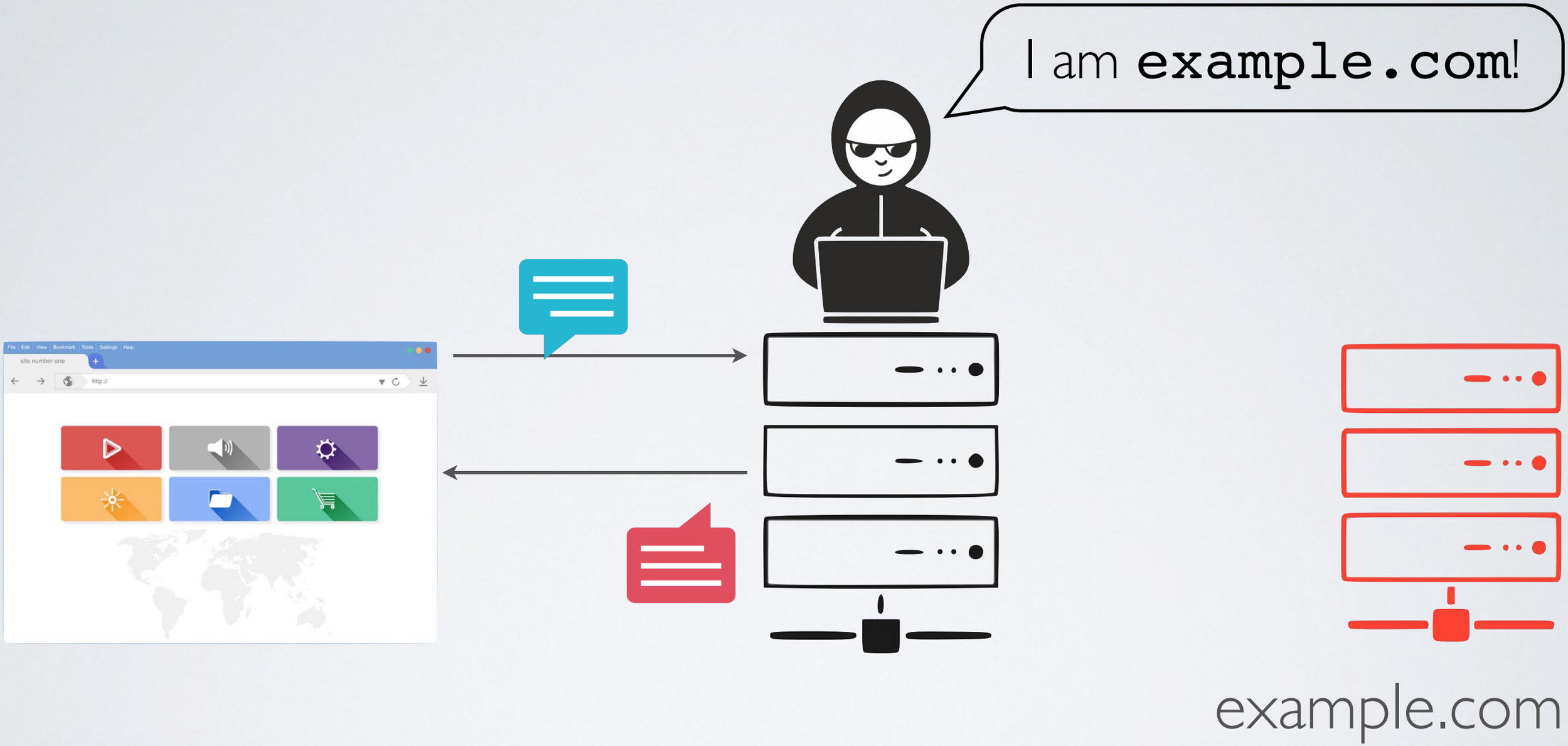


Do you trust the network?



- Threat 1 : an attacker **can eavesdrop** messages sent back and forth

Do you *really* trust the network?



- Threat 2 : an attacker **can tamper with** messages sent back and forth

Confidentiality and Integrity

- Threat 1 : an attacker **can eavesdrop** messages sent back and forth

  Confidentiality: how do exchange information secretly?

- Threat 2 : an attacker **can tamper** messages sent back and forth

  Integrity: How do we exchange information reliably?

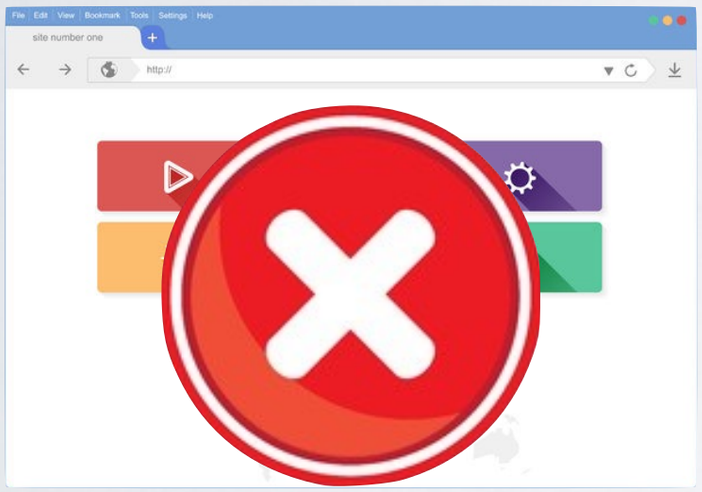
Generic solution - HTTPS

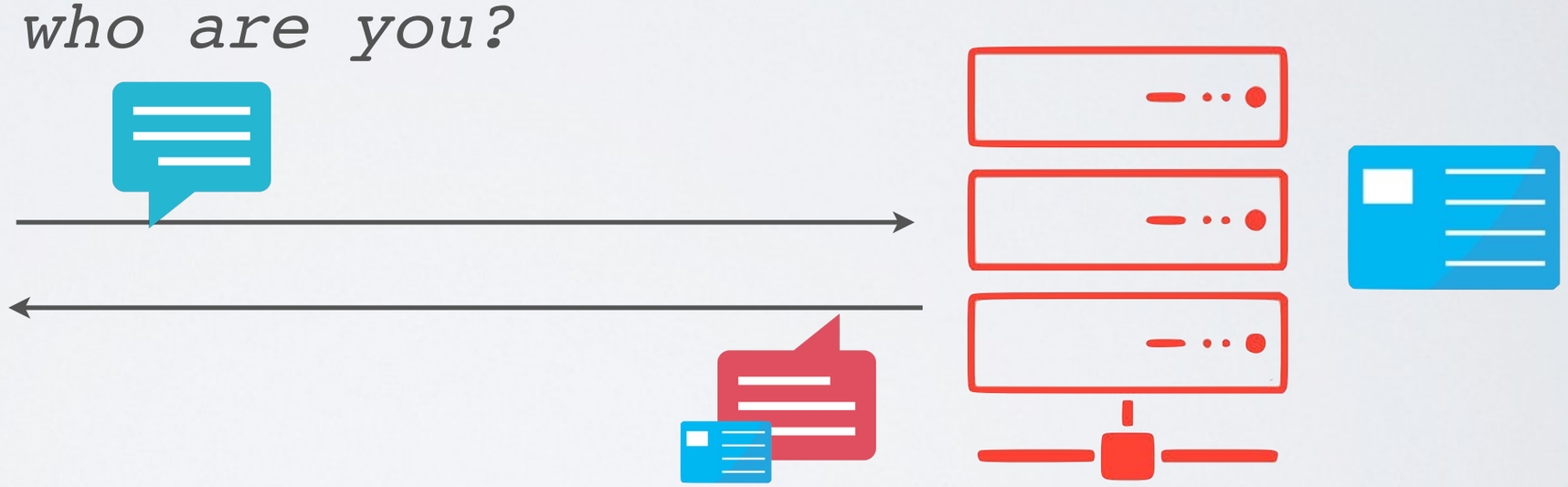
✓ HTTPS = HTTP + TLS

➡ Transport Layer Security (TLS previously known as SSL) provides

- **confidentiality:** end-to-end secure channel
- **integrity:** authentication handshake

Generating and using (self-signed) certificates





Self-signed certificates are not trusted by your browser



This Connection is Untrusted

You have asked Firefox to connect securely to **www.domainname.tld** but we can't confirm that your connection is secure.

Normally, when you try to connect securely, sites will present trusted identification to prove that you are going to the right place. However, this site's identity can't be verified.

What Should I Do?

If you usually connect to this site without problems, this error could mean that someone is trying to impersonate the site, and you shouldn't continue.

Get me out of here!

▸ **Technical Details**

▾ **I Understand the Risks**

If you understand what's going on, you can tell Firefox to start trusting this site's identification. **Even if you trust the site, this error could mean that someone is tampering with your connection.**

Don't add an exception unless you know there's a good reason why this site doesn't use trusted identification.

Add Exception...





Your connection is not private

Attackers might be trying to steal your information from **bitbucket.org** (for example, passwords, messages, or credit cards).

Hide advanced

Reload

bitbucket.org normally uses encryption to protect your information. When Chrome tried to connect to bitbucket.org this time, the website sent back unusual and incorrect credentials. Either an attacker is trying to pretend to be bitbucket.org, or a Wi-Fi sign-in screen has interrupted the connection. Your information is still secure because Chrome stopped the connection before any data was exchanged.

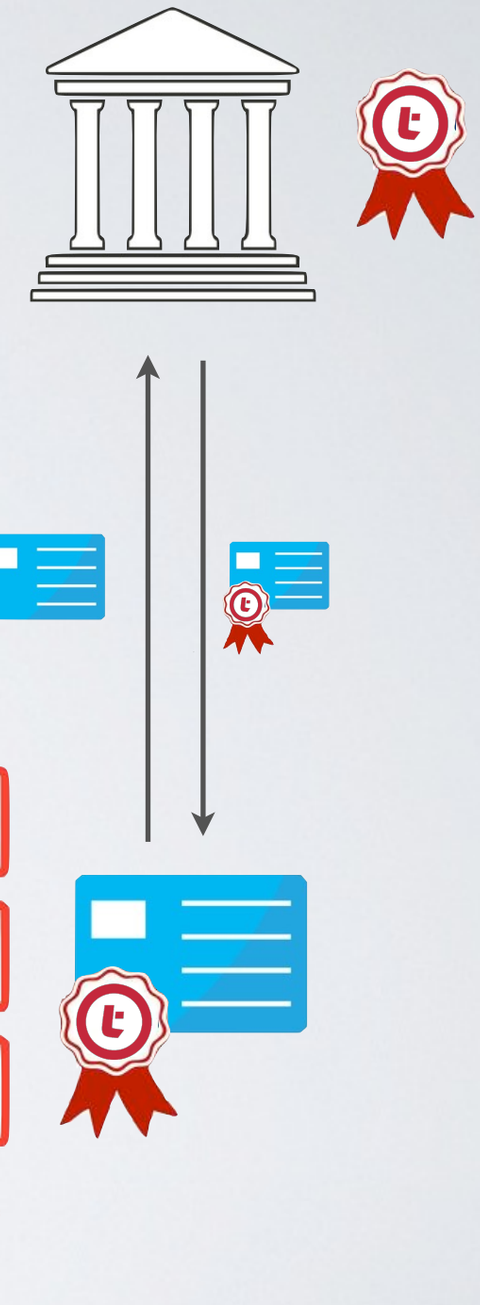
You cannot visit bitbucket.org right now because the website uses HSTS. Network errors and attacks are usually temporary, so this page will probably work later.

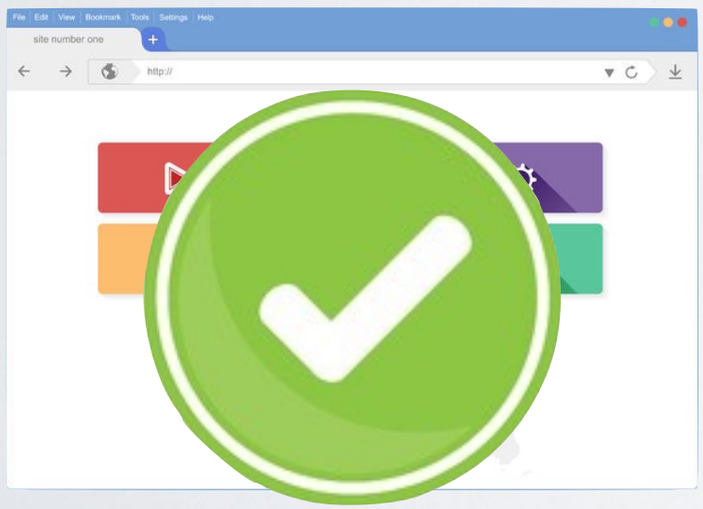
NET::ERR_CERT_DATE_INVALID



Signed Certificate

Certificate Authority (CA)

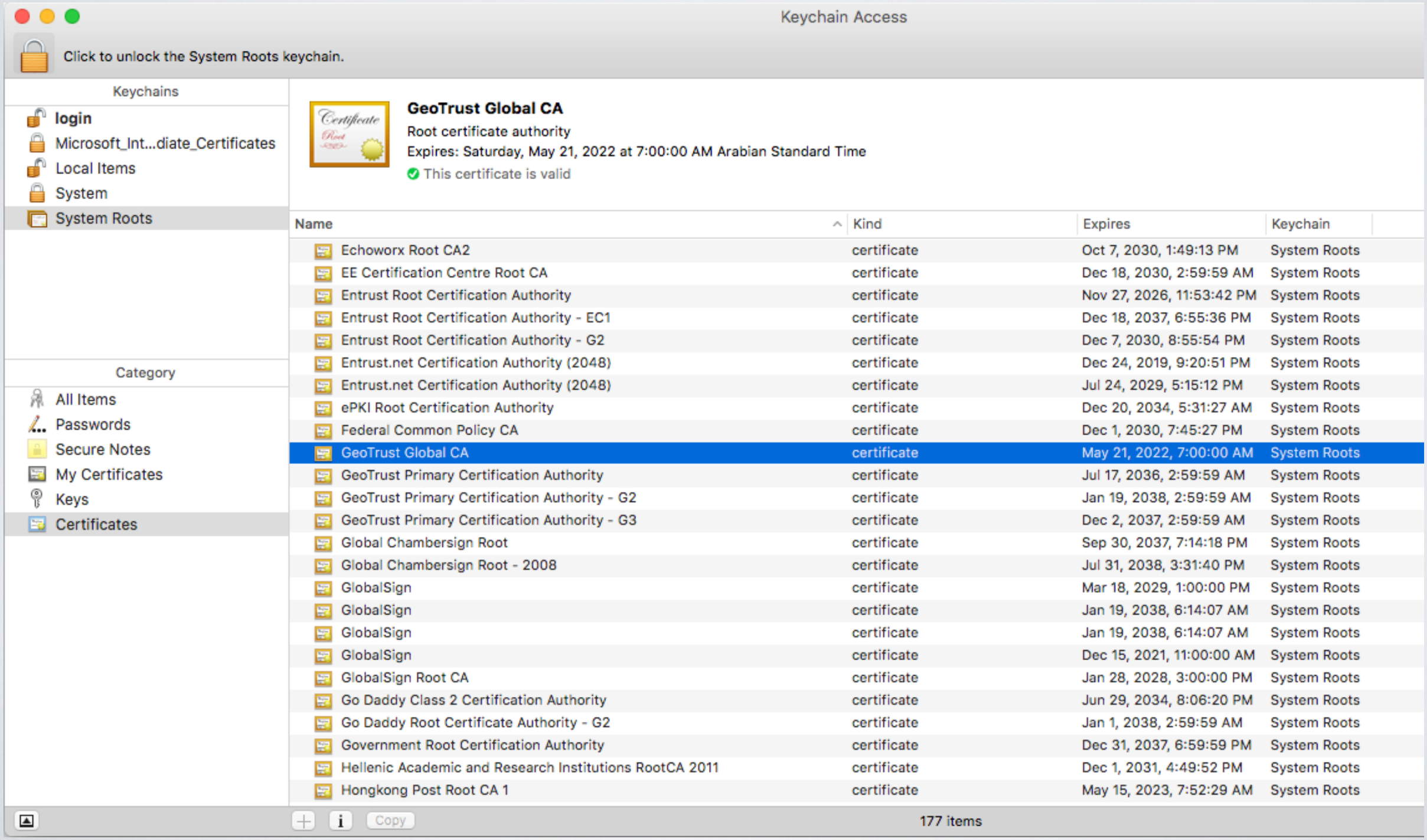




who are you?

I am example.com

Your browser trusts many CAs **by default**



Why and when using HTTPS?

HTTPS = HTTP + TLS

➡ TLS provides

- confidentiality: end-to-end secure channel
- integrity: authentication handshake

➡ HTTPS protects any data send back and forth including:

- login and password
- session ID

✓ **HTTPS everywhere**
HTTPS must be used during the entire session

Be careful of mixed content

Mixed-content happens when:

1. an HTTPS page contains elements (ajax, js, image, video, css ...) served with HTTP

2. an HTTPS page transfers control to another HTTP page within the same domain

◉ authentication cookie will be sent over HTTP

✓ browsers provide a mix-content protection now

Protection: `Secure` cookie flag

✓ The cookie will be sent over HTTPS exclusively

➡ Prevents authentication cookie from leaking in case of mixed-content

Protection:
HTTP Strict Transport Security (HSTS)

The server sends an HTTP response header

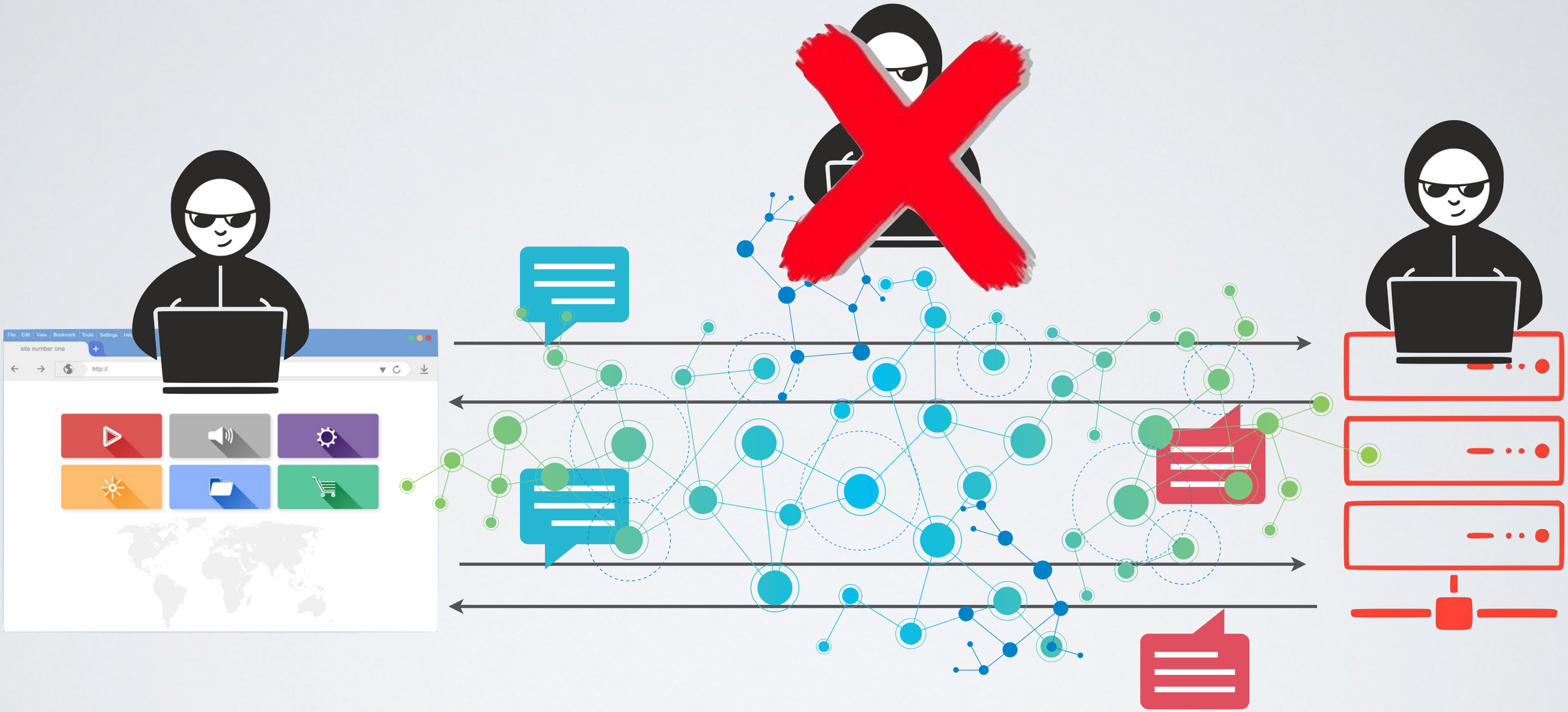
```
Strict-Transport-Security: max-age=31536000; includeSubDomains
```

➡ Forces browsers to communicate over HTTPS for future requests automatically

✓ **Benefits:** Reduces exposure to man-in-the-middle attacks (prevents HTTPS downgrade attacks such as SSL Stripping)

◉ **Limitation:** HSTS only takes effect after the first secure visit

Limitation of HTTPS

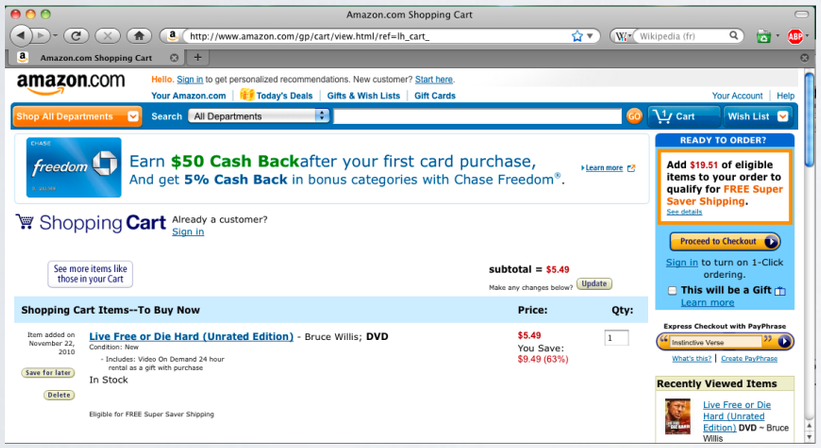


Problem

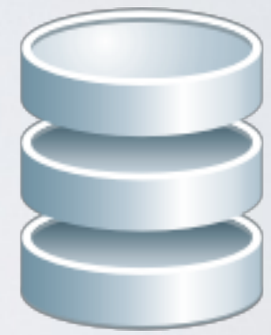
You have **absolutely no control** on the client and the network

Client Side

Server Side







Web Browser

Web Server

Database

Beyond HTTPS - attacking the web application

Frontend Vulnerabilities

- Cross-Site Scripting
- Cross-site Request forgery

Backend Vulnerabilities

- Incomplete Mediation
- SQL injection

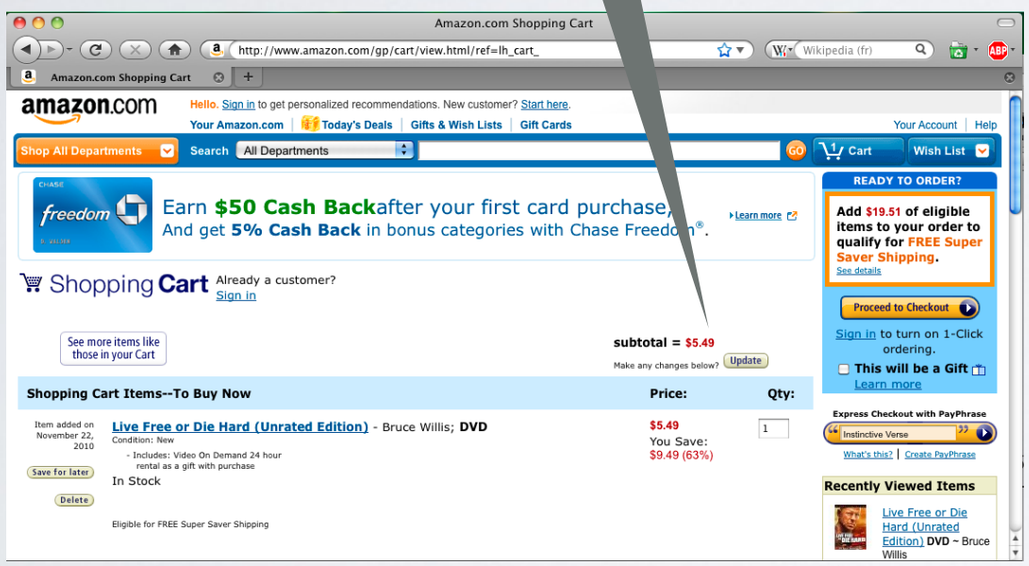
A01 Broken Access Control

Incomplete Mediation

The Shopping Cart Attack

The total is calculated by a script on the client

The order is generated based on the request



order=(#2956,10, 1, 10)

Thank you for your order!



Client Trusted Domain

Server Trusted Domain

* *Notice that Amazon is* ***not*** *vulnerable to this attack*

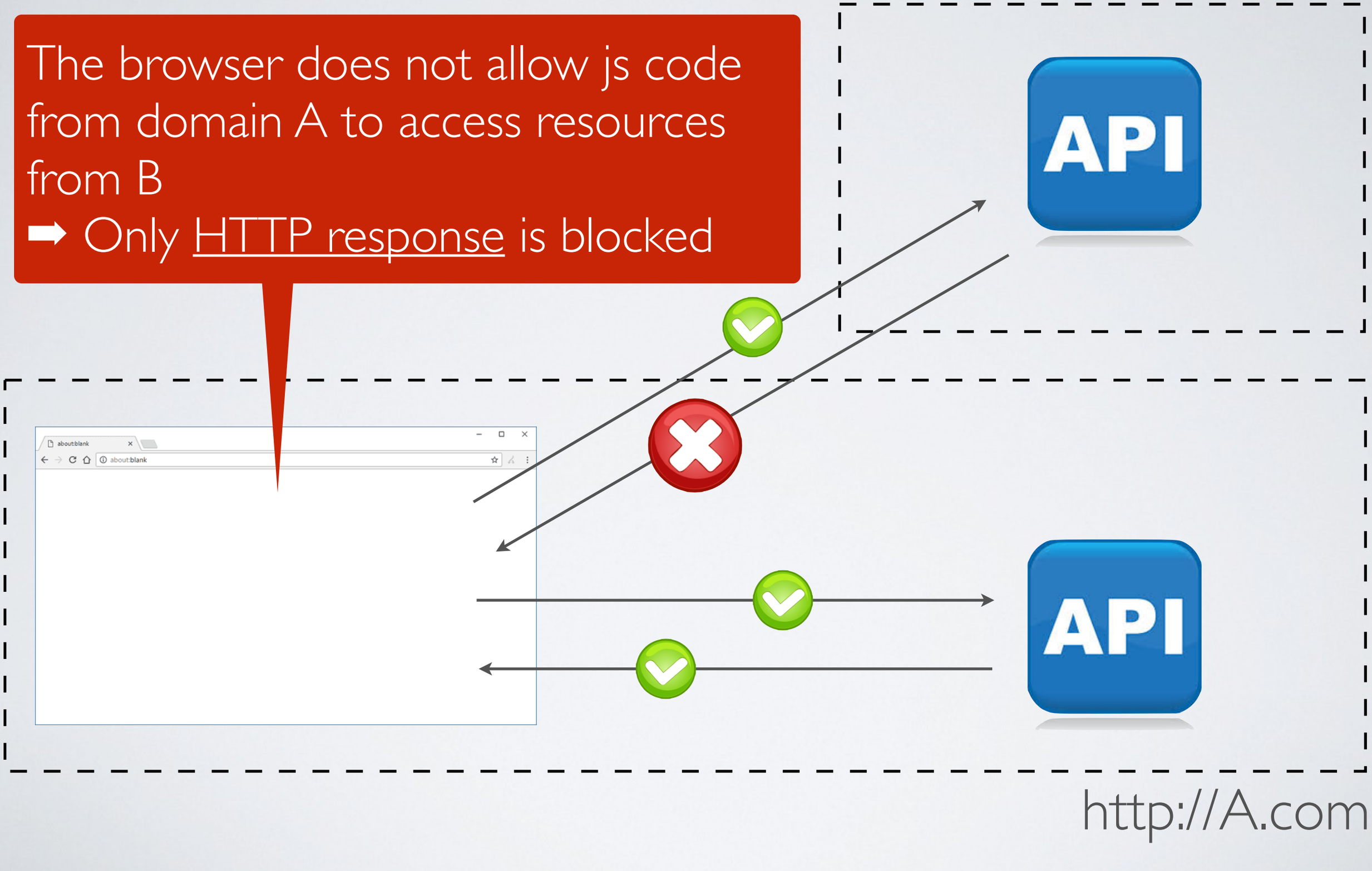
A01 Broken Access Control

Cross-Site Request Forgery

Ajax requests across domains

The browser does not allow js code from domain A to access resources from B

➡ Only HTTP response is blocked



Same origin policy

➡ **Resources must come from the same domain (protocol, host, port)**

Elements under control of the same-origin policy

- Ajax/Fetch requests
- Form actions

Elements **not** under control of the same-origin policy

- Javascript scripts
- CSS
- Images, video, sound

Examples

	client	server
same protocol, port and host	`http://example.com`	`http://example.com`
	`http://user:pass@example.com`	`http://example.com`
top-level domain	`http://example.com`	`http://example.org`
host	`http://example.com`	`http://other.com`
sub-host	`http://www.example.com`	`http://example.com`
sub-host	`http://example.com`	`http://www.example.com`
port	`http://example.com:3000`	`http://example.com`
protocol	`http://example.com`	`https://example.com`

[digression] relaxing the same-origin policy

- Switch to the superdomain with javascript
  `www.example.com` can be relaxed to **`example.com`**
- iframe
- JSONP
- Cross-Origin Resource Sharing (CORS)

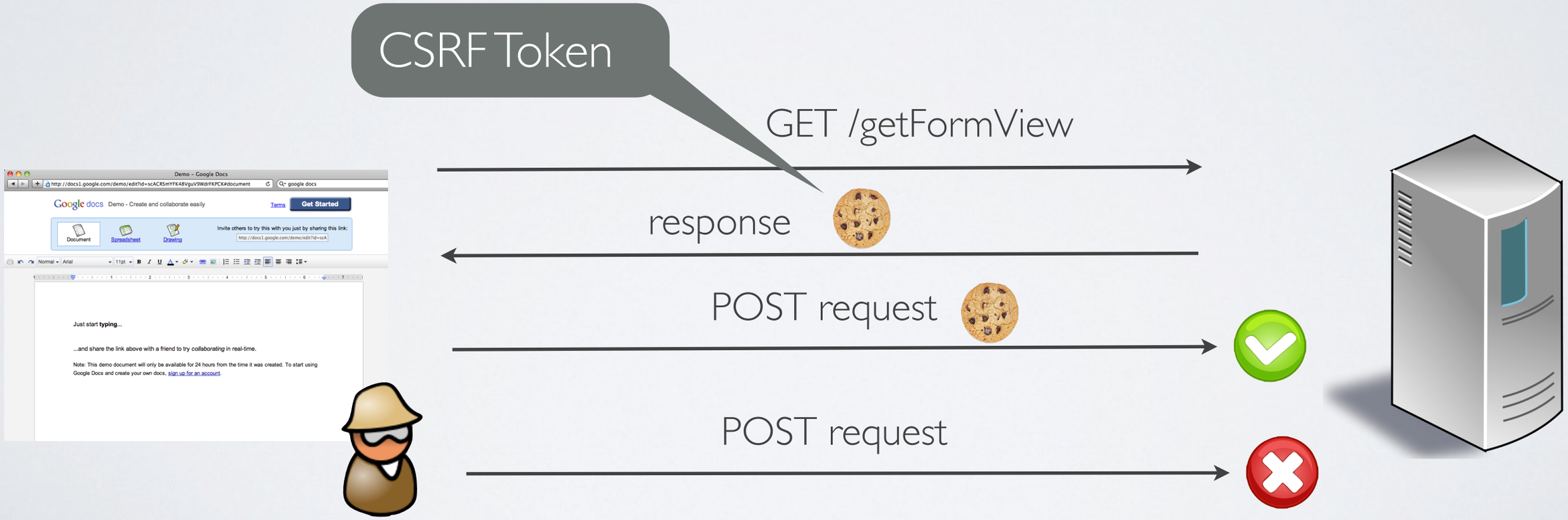
Problem

➡ An attacker can executes unwanted but yet authenticated actions on a web application by either

- setting up a malicious website with cross-origin requests
- or by injecting malicious urls into the page

Generic solution - CSRF tokens

✓ Protect legitimate requests with a CSRF token



`SameSite` cookie flag

✓ The cookie will be not be sent over cross-site requests

➡ Prevents forwarding the authentication cookie over cross-origin requests (cross-site request forgery)

A03 Injection

SQL Injection

Problem

- ➡ An attacker can inject SQL/NoSQL code
  - Retrieve, add, modify, delete information
  - Bypass authentication

Checking password

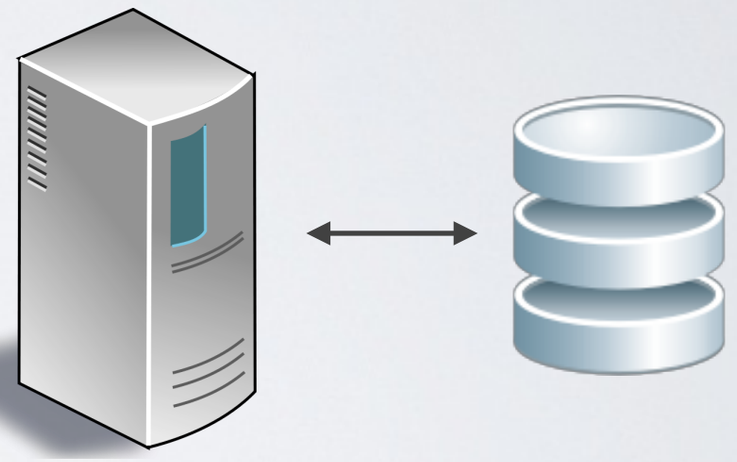
`signin.html`



`/signin/`

name=Alice&pwd=pass4alice

Access Granted!



SQL Injection

```
db.run("SELECT * FROM users
WHERE USERNAME = '" + username + "'
  AND PASSWORD = '" + password + "'"
```

```
username: alice
password: pas lice
```

```
blah' OR '1'='1
```

NoSQL Injection

```
db.find({   username: username,
            password: password    });
```

```
username: alice
password: pas lice
```

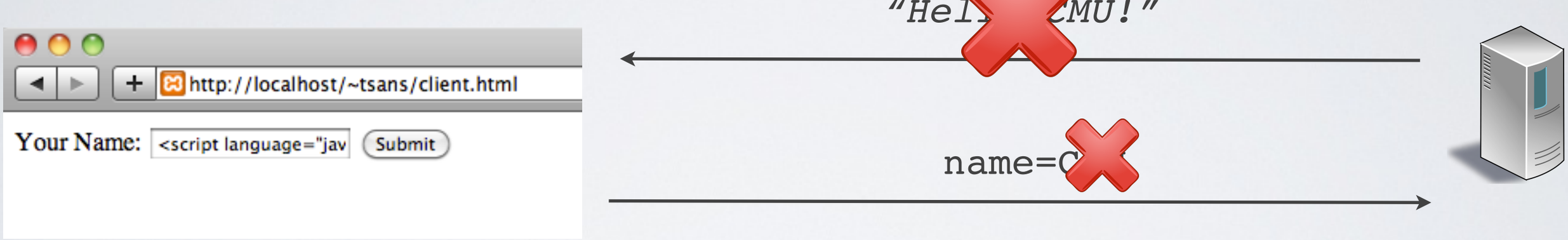


```
{gt: ""}
```

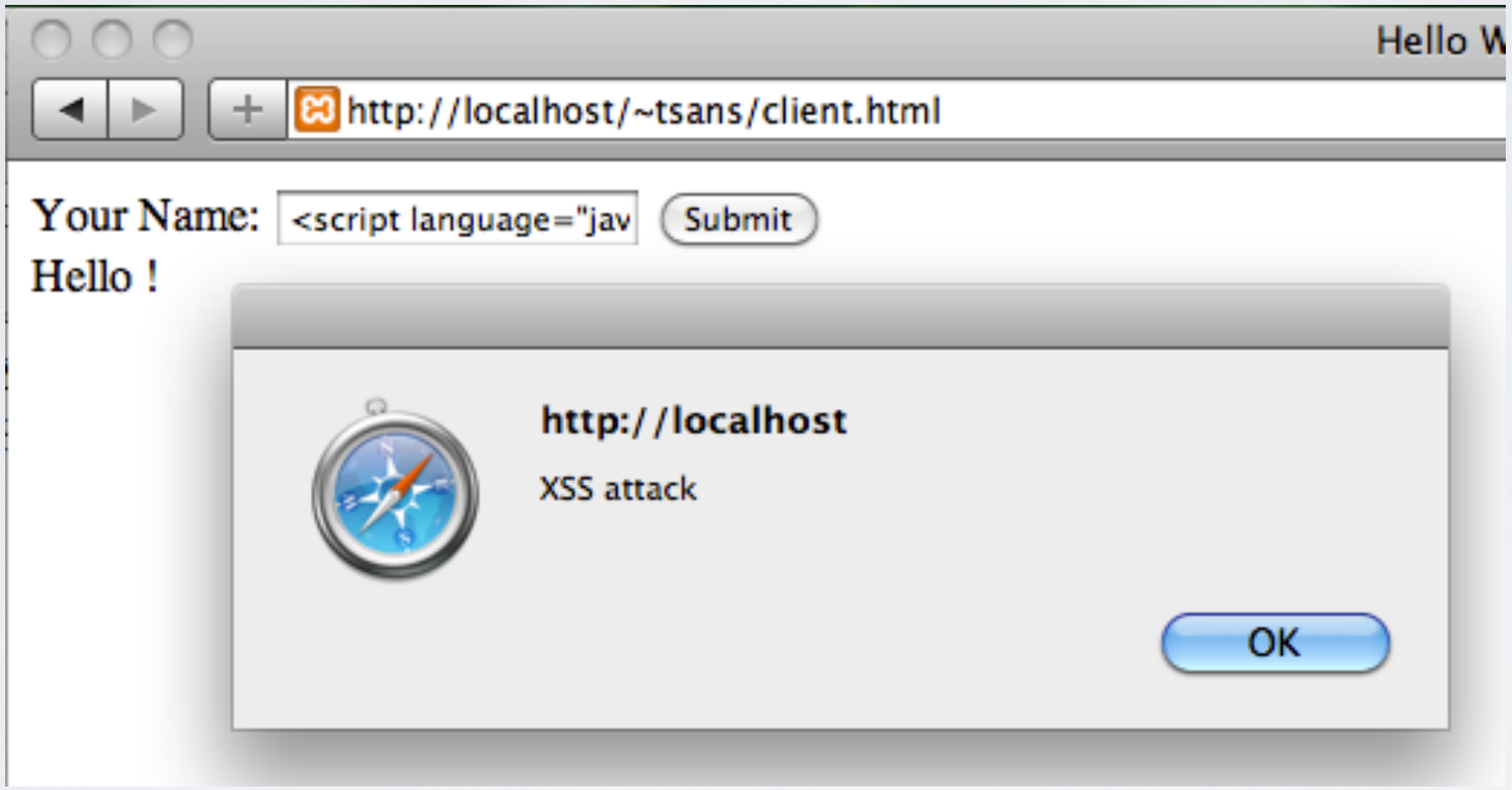
A03 Injection

Cross-Site Scripting (XSS)

Cross-Site Scripting Attack (XSS attack)



XSS Attack = Javascript Code Injection

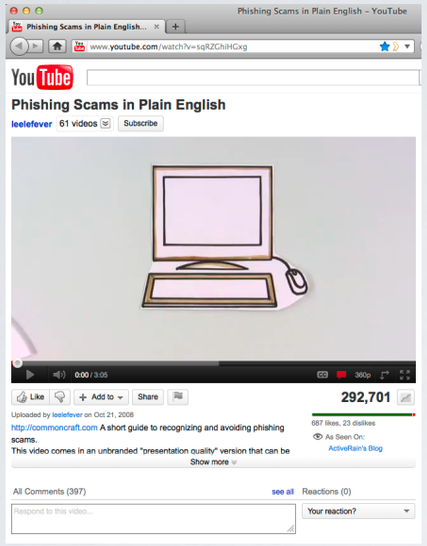


Problem

➡ An attacker can inject **arbitrary javascript code** in the page that will be executed by the browser

- **Inject illegitimate content** in the page (same as content spoofing)
- **Perform illegitimate HTTP requests** through Ajax (same as a CSRF attack)
- **Steal Session ID** from the cookie
- **Steal user's login/password** by modifying the page to forge a perfect scam

Forging a perfect scam



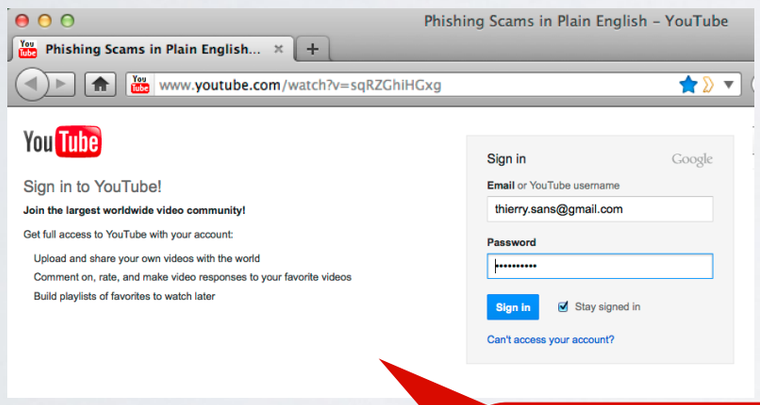


GET /?videoid=527

<html ...

comment = “<script> ...







GET /?videoid=527

<html ...

login=Alice&password=123456







The script contained in the comments modifies the page to look like the login page!

* *Notice that Youtube is* ***not*** *vulnerable to this attack*

It gets worst - XSS Worms

Spread on social networks

- Samy targeting MySpace (2005)
- JTV.worm targeting Justin.tv (2008)
- Twitter worm targeting Twitter (2010)

Variations on XSS attacks

- **Reflected XSS**
  Malicious data sent to the backend are immediately sent back to the frontend to be inserted into the DOM

- **Stored XSS**
  Malicious data sent to the backend are store in the database and later-on sent back to the frontend to be inserted into the DOM

- **DOM-based attack**
  Malicious data are manipulated in the frontend (javascript) and inserted into the DOM

Solution

- ✓ Data inserted in the DOM must be validated

Protection: `HttpOnly` cookie flag

✓ The cookie is not readable/writable from the frontend

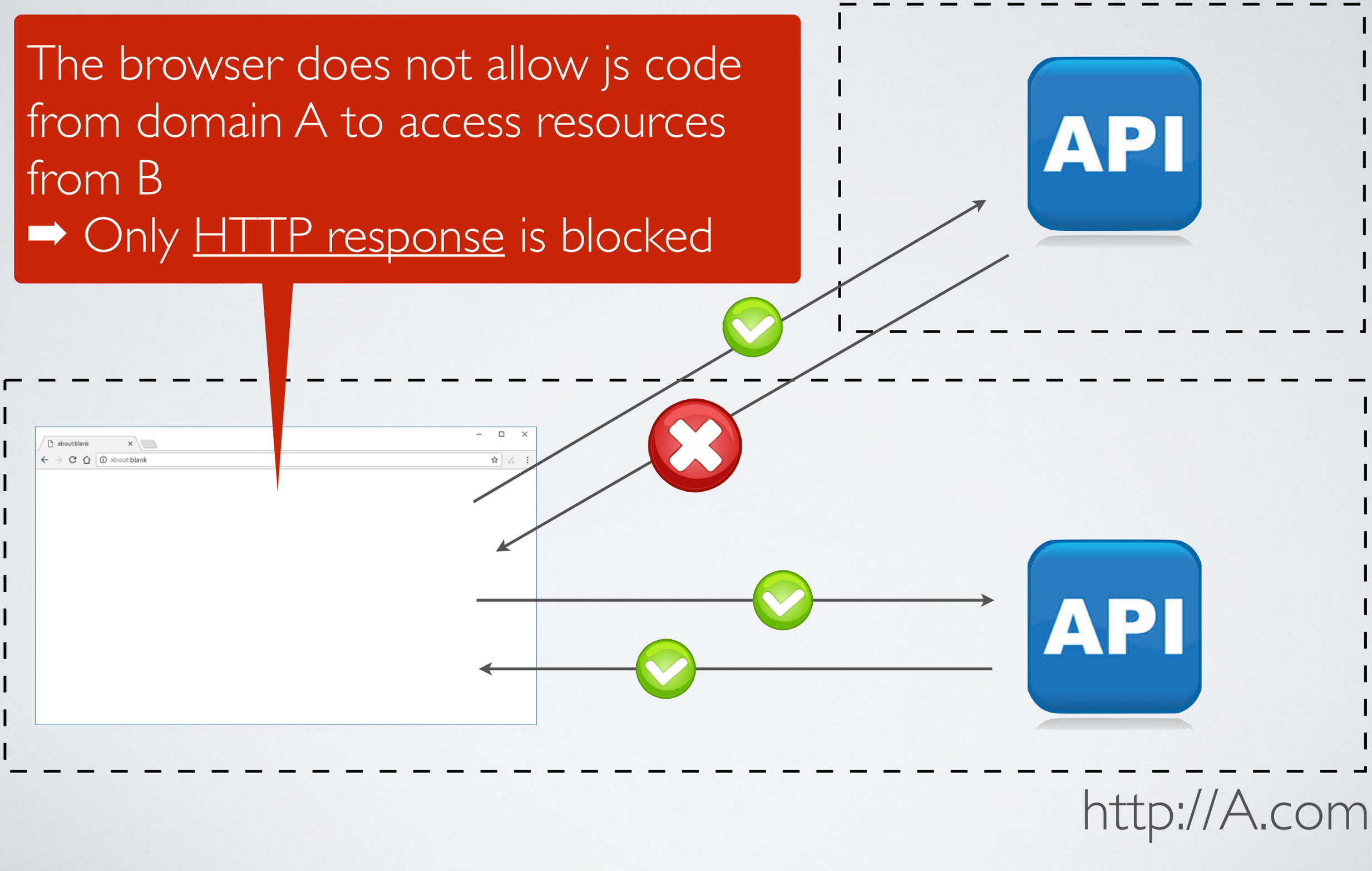
➡ Prevents the authentication cookie from being leaked when an XSS attack (cross-site scripting) occurs

Cross-Site Request Forgery

Ajax requests across domains

The browser does not allow js code from domain A to access resources from B

➡ Only HTTP response is blocked



Same origin policy

➡ **Resources must come from the same domain (protocol, host, port)**

Elements under control of the same-origin policy

- Ajax/Fetch requests
- Form actions

Elements **not** under control of the same-origin policy

- Javascript scripts
- CSS
- Images, video, sound

Examples

	client	server
same protocol, port and host	`http://example.com`	`http://example.com`
	`http://user:pass@example.com`	`http://example.com`
top-level domain	`http://example.com`	`http://example.org`
host	`http://example.com`	`http://other.com`
sub-host	`http://www.example.com`	`http://example.com`
sub-host	`http://example.com`	`http://www.example.com`
port	`http://example.com:3000`	`http://example.com`
protocol	`http://example.com`	`https://example.com`

[digression] relaxing the same-origin policy

- Switch to the superdomain with javascript
  `www.example.com` can be relaxed to **`example.com`**
- iframe
- JSONP
- Cross-Origin Resource Sharing (CORS)

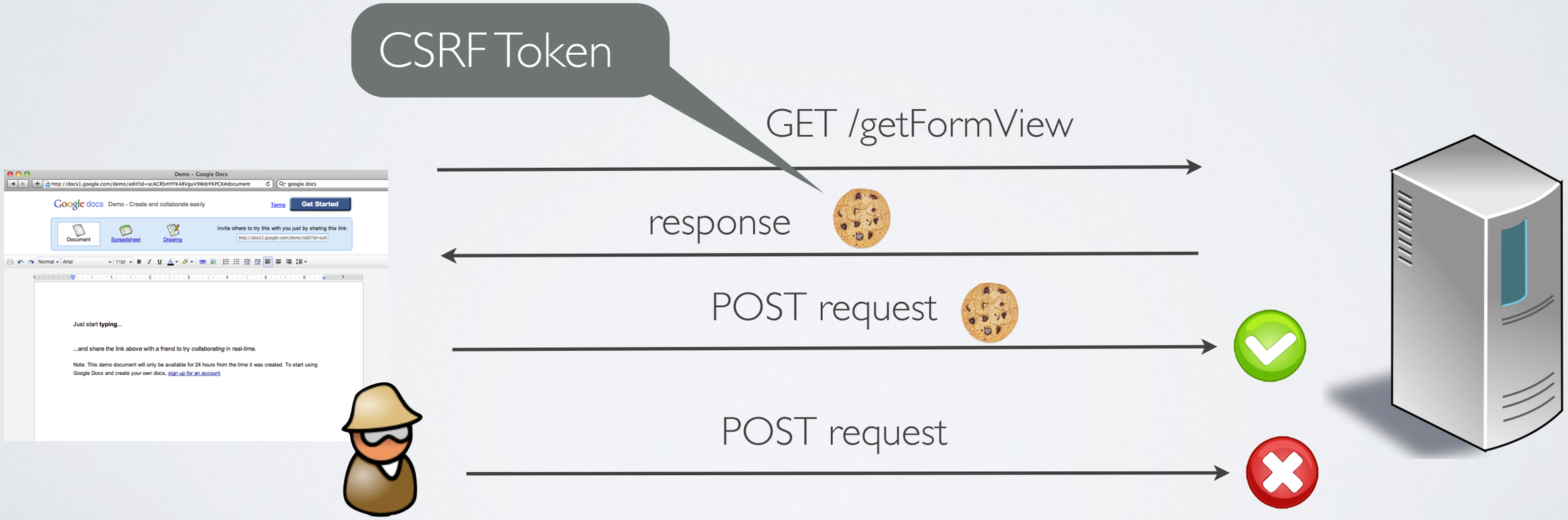
Problem

➡ An attacker can executes unwanted but yet authenticated actions on a web application by either

- setting up a malicious website with cross-origin requests
- or by injecting malicious urls into the page

Generic solution - CSRF tokens

✓ Protect legitimate requests with a CSRF token



`SameSite` cookie flag

✓ The cookie will be not be sent over cross-site requests

➡ Prevents forwarding the authentication cookie over cross-origin requests (cross-site request forgery)

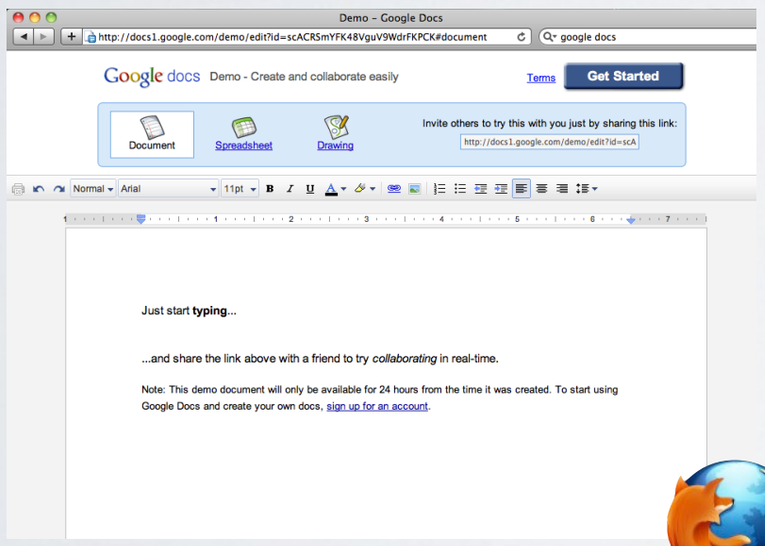
Conclusion

Conclusion

You have **absolutely no control** on the client

Client Side

Server Side

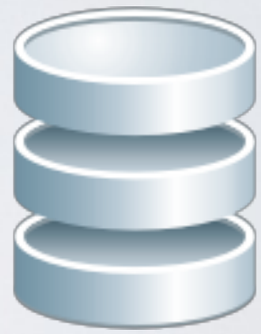




Web Browser



Web Server



Database

A01 - Broken Access Control

- **Problem:** Users can access data or perform actions beyond their intended permissions
- **Impact:** Unauthorized data disclosure, modification, deletion, or privileged actions

Common vulnerabilities :

- **Least privilege violations:** Excessive permissions
- **Access control bypass:** Manipulating URLs or API requests
- **Insecure direct object reference (IDOR):** Accessing other users' resources by changing ID
- **Privilege escalation:** Gaining unauthorized admin access
- **Token manipulation:** Tampering with JWTs or cookies
- **Misconfiguration:** Missing API authorization or overly permissive CORS

Preventing Broken Access Control

- ◉ **The Frontend is not trusted:** data coming from the frontend cannot be trusted
- ✓ **The backend is the only trusted domain:** sensitive operations must be done on the backend
- ✓ **Deny by default:** Explicitly grant access only when authorized

- **Least privilege:** Grant users only the permissions they need
- **Server-side enforcement:** Never rely on client-side access controls
- **Check every request:** Validate authorization for every resource and operation
- **Secure sessions:** Protect tokens and properly invalidate sessions
- **Restrict CORS:** Allow only explicitly trusted origins
- **Test authorization:** Verify that users cannot access other users' resources or privileged functions

A02 - Security Misconfiguration

- **Problem:** Insecure configuration of applications, servers, or cloud services that introduces vulnerabilities
- **Impact:** Unauthorized access, data exposure, or system compromise

Common vulnerabilities:

- **Default credentials:** Unchanged passwords or accounts
- **Unnecessary features:** Exposed ports, services, or debugging tools
- **Insecure permissions:** Misconfigured cloud storage or access controls
- **Information leakage:** Verbose error messages or stack traces
- **Missing security settings:** Disabled protections or missing HTTP security headers
- **Outdated configurations:** Legacy or insecure settings retained after upgrades

Preventing Security Misconfiguration

- **Secure by default:** Apply hardening guidelines across the entire stack
- **Minimize attack surface:** Disable unnecessary features, services, and accounts
- **Automate configuration:** Use consistent, repeatable configurations across environments
- **Keep configurations updated:** Review security settings after patches and upgrades
- **Enforce isolation:** Segment components and restrict cloud permissions
- **Secure credentials:** Avoid hardcoded secrets; use short-lived credentials
- **Validate security:** Automate configuration checks, restrict error details, and enforce security headers

A03 - Software Supply Chain Failures

- **Problem:** Vulnerabilities or compromises in third-party components, development tools, or software build and distribution processes
- **Impact:** Malicious code injection, application compromise, or widespread supply chain attacks

Common vulnerabilities:

- **Vulnerable dependencies:** Outdated, unpatched, or unsupported libraries and frameworks
- **Untrusted components:** Malicious or compromised third-party packages
- **Dependency blind spots:** Untracked direct or transitive dependencies
- **Insecure CI/CD:** Compromised build pipelines, repositories, or developer tools
- **Weak access controls:** Excessive privileges and lack of separation of duties
- **Poor change management:** Untracked changes to code, tools, or dependencies

Preventing Software Supply Chain Failures

- **Track dependencies:** Maintain a Software Bill of Materials (SBOM), including transitive dependencies
- **Monitor vulnerabilities:** Continuously scan components and track CVEs
- **Patch regularly:** Update vulnerable dependencies and development tools
- **Use trusted sources:** Prefer official repositories and signed packages
- **Secure CI/CD:** Enforce MFA, least privilege, code reviews, and separation of duties
- **Ensure integrity:** Sign builds, verify provenance, and protect artifacts
- **Control changes:** Track modifications to repositories, pipelines, and dependencies
- **Limit exposure:** Remove unused components and use staged deployments

A04 - Cryptographic Failures

- **Problem:** Weak or improperly implemented cryptography exposes sensitive data

- **Impact:** Data disclosure, credential theft, or compromised communications

Common vulnerabilities:

- **Insufficient Transport protection:** sensitive data transmitted without authentication and encryption
- **Password storage:** weak or unsalted password hashing

Preventing Cryptographic Failures

- **Encrypt HTTP session:** Protect sensitive data at rest and in transit (TLS 1.2+, preferably TLS 1.3)

- **Enforce secure communication:**
  - Valid certificates
  - Use `secure` cookie flag with authentication cookie
  - Enable HTTP Strict Transport Security (HSTS)

- **Hash passwords securely:** Use salted, slow password hashing (Argon2id, scrypt)

A05 - Injection

- ➡ **Problem:** Untrusted user input is interpreted as executable commands or queries
- ◉ **Impact:** Unauthorized data access, modification, deletion, or arbitrary code execution

Common vulnerabilities:

- **SQL/NoSQL injection:** Manipulating database queries
- **Command injection:** Executing arbitrary OS commands
- **Other injections:** LDAP, ORM, or expression language injection
- **Unsafe input handling:** Concatenating untrusted data into queries or commands
- **Insufficient validation:** Failing to validate or sanitize user input

Preventing Injection

✓ **Key principle:** Treat all user input as data, never as executable code

- **Validate input:** Apply strict server-side validation
- **Separate data from commands:** Never concatenate untrusted input into executable queries
- **Use parameterized queries:** Prefer prepared statements and safe APIs
- **Escape when necessary:** Use interpreter-specific escaping when parameterization is impossible

References

- OWASP Top 10
  https://owasp.org/www-project-top-ten/

- Mozilla Secure Coding Guideline
  https://wiki.mozilla.org/WebAppSec/Secure_Coding_Guidelines

- Node Express - Production Best Practices: Security
  https://expressjs.com/en/advanced/best-practice-security.html